

Property Tax Checker

Upload a workbook. We look up every property's tax status across any county portal in the country.
Download the same workbook, now with answers.

15+

U.S. states supported

Any

county portal, even unseen ones

100%

read-only — never pays a bill

1 file

in, same file out, structure
preserved

AGENDA

What we'll cover

01 The problem

Manual portal-by-portal tax tracking across 15+ states doesn't scale.

02 The product

Excel in → AI portal lookup → Excel out. One file, every property answered.

03 How it works

Four steps: classify the portal, find the property, read the bill, write the answer.

04 Architecture

Next.js frontend on Vercel, Python worker on Cloud Run, Skyvern for the browser, Firestore for state.

05 What makes it smart

Vision-based AI handles portals it has never seen, and writes itself a playbook for next time.

06 Cost & operations

What we pay each provider and the guardrails that keep it safe.

07 What's next

Roadmap from Excel workflow to live dashboard.

01 · THE PROBLEM

Property taxes don't scale by hand

Pontus owns property across **15+ states and dozens of counties**. Each county has its own tax portal — different URL, different layout, different login flow, different way to show whether a bill is paid.

An analyst opening every URL by hand, finding the property, reading the balance, and typing it into a spreadsheet is **hours of work per portfolio refresh**. Doing it monthly for hundreds of properties is unsustainable.

Worse, the data ages fast. The minute the spreadsheet is saved, a new bill could be posted, a payment cleared, or a delinquency penalty applied.

Why every portal is different

- Government portals run on **5+ vendor platforms** (Grant Street, Aumentum, PublicAccessNow, Tyler Technologies, Pacific Blue, plus one-offs)
- Tax year semantics vary: Florida is calendar-year, California is fiscal-year (2025-26), Texas bills in October
- Some portals deep-link to the property; most don't — you start at a search box
- Discounts, installment plans, exemptions: a property paid in full often shows less than the billed amount — and that's **correct**

02 · THE PRODUCT

One workflow. One file. One answer per property.

1. Upload

Drag the Excel tracker onto the web app. Any layout, any state. Headers are detected by meaning, not column letter.

2. Read

For each property, an AI browser agent opens the row's portal URL, classifies the page (deep link / search / multi-step / PDF / blocked), and navigates to the right account.

3. Verify

Owner name, address, parcel — all cross-checked before anything gets recorded. If the page doesn't match the property, it's flagged, not guessed.

4. Download

The original Excel file is returned with a new '<Month> Update' column filled in — formulas, formatting, merged headers all preserved.

The crucial design choice: the spreadsheet stays the source of truth. Analysts keep their workflow. We just make the lookup step disappear.

03 · THE SMART PART

We classify any portal we've never seen into one of 7 shapes

Type	Pattern	What we do
A	Direct account page	URL deep-links straight to the property. Read and move on.
B	Search form	Type an account #, parcel, or address — handle the results list.
C	Multi-step flow	Disclaimer page, roll-type selector, then the search. Click through.
D	Year selector	Some portals pin the tax year in the URL. We confirm the year matches.
E	Blocked	Login wall, CAPTCHA, paywall. We mark NEEDS_REVIEW — never bypass.
F	PDF-only	Some counties expose only PDF bills. We download, OCR if needed, parse.
G	Split assessor/collector	Texas-style: appraisal district and tax office are different sites.

If the vendor is brand new — we look at the page footer ('Powered by Grant Street Group', 'Tyler Technologies', etc.), solve it generically, and **write ourselves a playbook entry** for next time. The system gets smarter every run.

04 · ARCHITECTURE

Six pieces. Each does one thing well.

Layer	What it does	Where it runs
Frontend	Next.js 16 (App Router) + TypeScript + Tailwind + shadcn/ui	Vercel
API layer	Next.js API routes — upload, list runs, download, cancel	Vercel
Database	Firestore (NoSQL) — runs, rows, events, playbooks	Firebase
File storage	Workbook uploads + processed outputs	Firebase Storage
Worker	Python — orchestrator, intake, verify, validate, write-back	Google Cloud Run Job
Browser AI	Vision-based scraping agent that drives any portal	Skyvern (managed)

Why this stack? Every layer is pay-as-you-go and platform-native. Vercel + Firebase + Cloud Run are all part of the same standard Pontus tooling template — every internal tool we build sits on the same foundation, so engineers move between projects without re-learning the plumbing.

04B · DATA FLOW

What happens between upload and download

01 User uploads .xlsx

Browser POSTs the file to the Next.js API on Vercel.

02 File lands in Firebase Storage

A new *tax_checker_runs* document is created with status **queued**, one sub-doc per property row.

03 Cloud Run Job picks up the run

The Python worker claims the next queued run, groups rows by portal domain, and opens browser sessions.

04 Skyvern drives the portals

For each row, the worker sends a structured task to Skyvern: 'open URL, classify, find account, return amount due'.

05 Verify + validate

Owner / parcel / address are cross-checked. Discount bands, delinquent-growth rules, year-match — all enforced before writing.

06 Write-back to Excel

openpyxl opens the original workbook, fills the canonical columns + a new '*June 2026 Update*' column, preserves every formula and merged header.

07 Download

The processed file is uploaded back to Firebase Storage; the UI gets a signed URL; the user clicks **Download**.

05 · UNDER THE HOOD

The engine — what each module does

<code>intake.py</code>	Detect headers across any workbook. Florida uses stacked header rows; California uses single. Map columns by synonym matching, not column letter.
<code>identifiers.py</code>	Normalize dirty account / parcel data. Strip '#', dashes, leading zeros; split multi-account cells; try each candidate in order until something hits.
<code>taxonomy.py</code>	Classify whatever the portal landed on into one of the 7 shapes (A–G).
<code>playbooks.py</code>	Vendor-specific quirks library — recognizes Grant Street, Aumentum, PublicAccessNow, Pacific Blue, Tyler. Auto-extends with new vendors as we meet them.
<code>skyvern_runner.py</code>	The bridge to Skyvern. One workflow per taxonomy type, parameterized by URL + identifiers.
<code>verify.py</code>	Ownership + address cross-check before any extraction. Fuzzy match on owner names (county data entry is messy); exact match on parcel.
<code>validate.py</code>	The discount-band, delinquent-growth, no-silent-erasure rules. A scraped \$0 NEVER overwrites a real value.
<code>writeback.py</code>	openpyxl writes the answers back. Cells with formulas are protected — Florida's Total column W and the SUM row are untouched.
<code>orchestrator.py</code>	The conductor. Resumable per-row, polite rate-limiting per portal, on-the-spot cancel.

06 · SAFETY GUARDRAILS

What the system will not do, by design

Hard rules

- **Absolute read-only** on portals. Never add to cart, checkout, pay, enroll in installment plans, create accounts, or log in to payment accounts.
- **Never auto-bypass** a CAPTCHA wall or eligibility gate. If genuinely blocked → mark NEEDS_REVIEW.
- **No silent erasure.** A scraped \$0 NEVER overwrites a real existing value. Parse glitches must not delete data.
- **Wrong-property protection.** Owner, address, or parcel must match before anything is extracted.

Confidence model

Every row gets a confidence tag:

HIGH — owner + parcel verified, payment details read directly.

MEDIUM — fuzzy match or proof section unavailable. Recorded with evidence trail.

LOW / NEEDS_REVIEW — anything inferred. Goes to the status column only, never into data columns.

The user always sees why we landed on each answer.

07 · WHAT IT COSTS TO RUN

Pay-as-you-go across four providers

Provider	Pricing model	Practical cost
Skyvern (browser AI)	Per-step pricing on the cloud plan — roughly \$0.10 / step. A typical property lookup runs in 36 steps	\$0.36 — \$0.60 / property
Vercel (frontend + API)	Hobby tier free for early use. Pro plan \$20 / month covers our scale comfortably (function invocations, bandwidth).	\$0 — \$20 / month
Firebase (database + storage)	Blaze pay-as-you-go. Firestore reads/writes are sub-cent per 1k. Storage measured in GB-months for our workbook sizes.	Pennies / month
Google Cloud Run (worker)	Pay for the seconds the job runs. The job sleeps between uploads, so we're billed only during active processing.	Pennies / upload

Bottom line for a 100-property portfolio refresh: the bulk of the bill is Skyvern's per-step pricing — roughly **\$30–\$60 of variable cost** per full run. Fixed infrastructure is under \$25/month. Compare that with one analyst-hour at any reasonable rate, and a refresh that used to take a full afternoon now costs less than lunch.

Skyvern pricing tiers are subject to change — confirm against the current cloud plan before quoting externally.

07B · WHY SKYVERN, SPECIFICALLY

The AI browser that handles portals it's never seen

Traditional scrapers (Playwright, Puppeteer, BeautifulSoup) need a script per portal. Five portals = five scripts. Fifty portals = a maintenance nightmare.

Skyvern uses **vision-based AI** — it looks at the page like a person would, reads the labels, and decides what to click. One workflow generalizes across vendors.

It also handles CAPTCHAs natively, supports persistent browser sessions, and exposes a clean HTTP API. We point at a URL, give it a schema for what we want extracted, and let it work.

What Skyvern handles for us

- Visual element detection — no CSS selectors to maintain
- CAPTCHA solving (many types)
- Multi-step navigation (disclaimers, search → results → detail)
- Schema-validated extraction — the LLM is forced into the shape we asked for
- Browser session reuse — polite to the same portal across many rows

08 · OPERATIONS & RESILIENCE

Built to survive the things that go wrong in production

Resumable per-row	If the worker crashes mid-run, the next execution picks up exactly where the last one left off. Nothing is re-paid for, nothing is lost.
Polite same-portal queuing	Rows on the same portal domain run sequentially, not in parallel. We never hammer a county tax office.
Cancel-on-the-spot	Hit cancel in the UI → in-flight Skyvern tasks abort within five seconds. No waiting out an 8-minute scrape.
Browser session reaping	Crashed runs are detected at startup and their orphaned browser sessions are closed before any new ones open — keeps us under Skyvern's session cap.
Service-account isolation	The frontend never touches Firestore directly; all writes go through server-side API routes. Security rules deny everything by default.
Append-history, never overwrite	Every property has a history sub-collection in Firestore. Past run results are queryable forever.

09 · THINGS THAT MAKE THIS FUN

Edge cases nobody thinks about until they break you

Florida row 4 has THREE accounts in one cell

#T815151/#T813795/#R444958 — we split, look each up, and only mark the row PAID if *all three* are paid.

4% discount in November is not a partial payment

Florida gives a 4%/3%/2%/1% early-pay discount by month. A property that paid \$4,974 of a \$5,128 bill in December isn't delinquent — it's smart. Our validation rules know this.

'2025 bill paid 03/2026' is normal

Florida bills are payable through March of the following year. Texas, California, and every other state have their own calendar. The system knows the rules per state.

Tangible vs real estate is not the same parcel

One Pontus URL decodes to `charlotte:tangible:<uuid>` — personal property tax, not real estate. The same address can have both rolls. We pick the right one.

Recently-acquired properties show the seller's name

Counties don't update owner records instantly. If owner doesn't match but parcel AND address both match, we proceed with medium confidence and flag the owner mismatch in the evidence trail.

'Total Payable: \$0.00' isn't proof of payment

It means no current balance — the actual receipt usually hides in a collapsed 'Recently Paid Bills' section. We expand it. (Or in fast-mode, we just record PAID and move on — that's the user's call.)

10 · WHAT'S NEXT

Roadmap

Live dashboard view

Today the output is Excel. Next: a real-time portfolio view in the web app, with status badges and per-property history charts pulled straight from Firestore.

Scheduled refresh

A daily Cloud Function that re-runs the latest workbook automatically and emails a diff: who became delinquent, who paid, what changed.

Appeals & assessment monitoring

Same engine, different schema — track assessed value over time and flag opportunities to appeal.

Coverage expansion

Each new state adds a few county playbook entries. The system is designed to handle 'never seen this before' on day one — every new portal we meet writes its own playbook.

Tighter integrations

Slack alert when a property goes delinquent. Webhook into the accounting system to reconcile payments automatically.

IN ONE SLIDE

Why this matters

Property tax tracking has been the kind of work that *has to be done* but nobody has time for. The portals are fragmented, the data ages fast, and the consequences of missing a delinquency are real money in penalties.

This tool turns the loop — **open URL, find property, read balance, type into spreadsheet** — from analyst hours into an upload and a download.

It's **generic across every state**. It's **safe by construction**: read-only on the portals, deny-by-default on the database, no silent erasure of existing data, and a confidence model on every row.

And it **gets smarter with every run** — each new vendor portal we see adds a playbook entry that future runs reuse.

One Excel file in. The same file, answered, out.

APPENDIX

Where things live

Code repo	Local: New Tax Project/ — Next.js at root, Python worker under worker/.
Live site	Vercel project tax-project-qso5 (env vars marked Sensitive).
Firebase	Project pontustax (Blaze tier). Bucket pontustax.firebaseio.com.
Cloud Run job	tax-checker-worker — deployed via gcloud run jobs deploy --source worker.
Skyvern	Cloud account, ~10 concurrent portals configured (effective ceiling is plan-dependent).
Tests	36 tests on synthetic workbook + dry-run pipeline (npm run worker:test).